

SAT 求解器的技术演进及其在软件验证中的应用

《计算机科学经典方法与理论》课程汇报

刘鼎琨，武汉大学计算机学院

April 1, 2026

目录

- 1 布尔可满足性问题概述
- 2 SAT 求解技术的国内外现状调研
- 3 典型应用场景与案例剖析
- 4 SAT 研究的热点与未来趋势
- 5 结论

1.1 SAT 问题的定义与基本形式

布尔可满足性问题 (**SAT**): 判定给定的布尔公式是否存在一组真值指派使其结果为真。

核心组件:

- 布尔变量: $X = \{x_1, \dots, x_n\}$, 取值 $x_i \in \{0, 1\}$
- 文字 (**Literal**): 变量 x_i 及其否定 $\neg x_i$
- 子句 (**Clause**): 文字的析取 ($\vee$), 如 $C = l_1 \vee \dots \vee l_k$

合取范式 (**CNF**): SAT 问题的标准输入格式, 即子句的合取 ($\wedge$):

$$F = \bigwedge_{j=1}^m C_j = \bigwedge_{j=1}^m \left(\bigvee_{i=1}^{k_j} l_{j,i} \right) \quad (1)$$

1.2 判定结果: SAT vs. UNSAT

求解器的核心输出是判定公式 F 的满足性。

可满足 (SAT):

- 存在真值指派 μ 使得 $F(\mu) = 1$
- μ 称为公式的一个模型
- 应用: 路径规划存在路径、硬件触发特定行为

不可满足 (UNSAT):

- 对于任意指派 μ , 都有 $F(\mu) = 0$
- 应用: 在形式化验证中, 证明系统是安全的

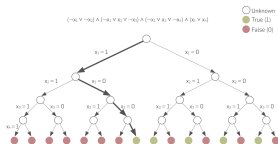


Figure: 一个 CNF 范式的搜索空间样例

1.3 典型的 SAT 变体

- **2-SAT**: 每个子句包含至多两个文字, 可在线性时间内求解。
- **3-SAT**: 每个子句包含三个文字, 标准的 **NP** 完全问题。
- **Max-SAT**: 优化版本, 极大化可满足子句数。
- **SMT (Satisfiability Modulo Theories)**: 在一阶逻辑上的扩展, 将特定领域的数学理论 T (如线性算术、位向量) 嵌入布尔骨架。

2.1 SAT 求解技术演进路线

从理论奠基到工业级应用的巨大跨越。

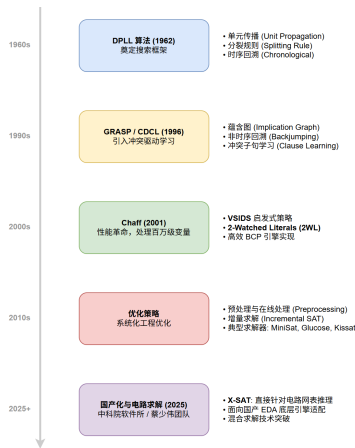


Figure: SAT 求解技术演进路线

2.2 DPLL 算法 (1962)

基于回溯的深度优先搜索框架。奠定了完备性 SAT 求解器的基础。

关键步骤：

- 单元传播 (**Unit Propagation**): 若存在单文字子句 (l), 则必须指派 $l = 1$ 。
- 分裂规则 (**Splitting Rule**): 选择一个未赋值变量 x , 分别探索 $x = 1$ 和 $x = 0$ 两个分支。

$$(\neg A \vee B) \wedge (\neg C \vee \neg B) \wedge (C \vee \neg B) \wedge (A \vee D)$$

• DPLL algorithm

- Decide : $A \rightarrow \text{true}$
- Propagate : $B \rightarrow \text{true}$
- Propagate : $C \rightarrow \text{false}$

$\Rightarrow$ Conflicting clause!

(all literals are false)

...*backtrack* on a decision

• DPLL algorithm

- Backtrack : $A \rightarrow \text{false}$
- Propagate : $D \rightarrow \text{true}$
- Decide : $B \rightarrow \text{false}$

$\Rightarrow$ Input is

SAT

by interpretation where
 $\{A \rightarrow \perp, B \rightarrow \perp, D \rightarrow T\}$

2.3 CDCL (Conflict-Driven Clause Learning)

现代求解器的基本逻辑，通过“记忆”机制提升效率。

核心技术 1: **GRASP** 算法 (20 世纪 90 年代末)

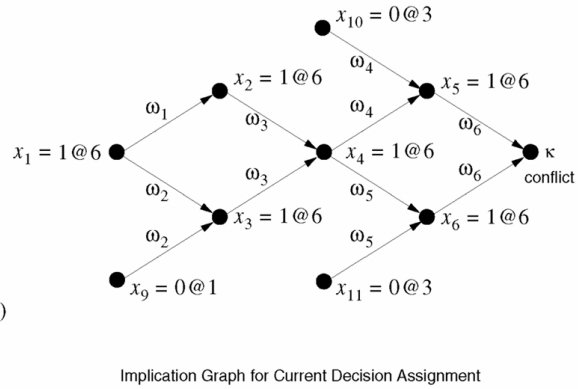
- 蕴含图 (**Implication Graph**): 描述决策层与赋值文字的逻辑推导关系。
- 子句学习: 当产生冲突时, 通过归结运算提取冲突根源约束:

$$C_{new} = (C_{conf} \setminus \{x\}) \vee (C_{ante} \setminus \{\neg x\}) \quad (2)$$

- 非时序回溯: 利用学习子句, 直接跳回到导致冲突的最早决策层。

Current Truth Assignment: $\{x_9 = 0@1, x_{10} = 0@3, x_{11} = 0@3, x_{12} = 1@2, x_{13} = 1@2, \dots\}$
 Current Decision Assignment: $\{x_1 = 1@6\}$

$\omega_1 = (\neg x_1 + x_2)$
 $\omega_2 = (\neg x_1 + x_3 + x_9)$
 $\omega_3 = (\neg x_2 + \neg x_3 + x_4)$
 $\omega_4 = (\neg x_4 + x_5 + x_{10})$
 $\omega_5 = (\neg x_4 + x_6 + x_{11})$
 $\omega_6 = (\neg x_5 + \neg x_6)$
 $\omega_7 = (x_1 + x_7 + \neg x_{12})$
 $\omega_8 = (x_1 + x_8)$
 $\omega_9 = (\neg x_7 + \neg x_8 + \neg x_{13})$
 $\dots$
 Clause Database



2.3 CDCL (Cont.)

核心技术 2: Chaff 求解器 (2001) ——走进工业界

- **VSIDS** 启发式变量选择: 动态调整变量“活跃度”分数 $S(l)$, 倾向于选择连接不同社区的桥变量。
- 双观测变量 (**2WL**): 优化状态传播, 省去繁琐的计数器更新。

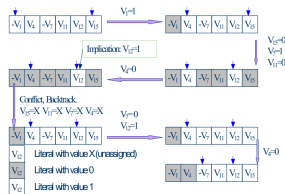


Figure: 2WL 的 BCP 过程示例

优化策略:

- 预处理 (如变量消除) 和 在线处理。

2.4 最新国内外研究现状

- 国际前沿：Armin Biere 团队研发的 **Kissat** 求解器，性能和证明生成保持领先。
- 国内突破：中科院软件所蔡少伟团队在 DAC 2025 上提出电路求解器 **X-SAT**。

直接以门级网表作为推理单元，处理乘加混合电路时性能较 Kissat 提升 35.94%。

3.1 建模方法：从现实问题到布尔语言

建模质量直接决定求解效率，核心技术包括：

- **位向量编码 (Bit-blasting)**: 将 w 位整数映射为 w 个布尔变量，利用门级逻辑（加法器、乘法器）构建约束。
- **序编码 (Order Encoding)**: 引入变量 $b_{x \leq i}$ ，利用单调性减少搜索分支，适用于资源调度。
- **电路级推理**: 直接保留门级网表语义，避免 Tseitin 变换引入冗余变量。

3.2 有界模型检测 (BMC)

核心原理：将有限状态系统 M 在步长 k 内进行循环展开。

判定公式 F_k [?]:

$$F_k = I(s_0) \wedge \bigwedge_{i=0}^{k-1} T(s_i, s_{i+1}) \wedge \bigvee_{i=0}^k \neg P(s_i) \quad (3)$$

- 结果为 **SAT**: 找到反例轨迹 (Bug)。
- 结果为 **UNSAT**: 证明系统在 k 步内绝对安全。

案例：红绿灯系统

- 初始状态 $I(s_0)$ 、迁移关系 T (红 $\rightarrow$ 绿 $\rightarrow$ 黄) 及互斥约束。

编码初始状态定义为红灯亮，即 $I(s_0) = R_0 \wedge \neg Y_0 \wedge \neg G_0$ 。其后逻辑建模中，要描述红绿灯转换的编码迁移关系 $T(s_i, s_{i+1})$ ，即红 $\rightarrow$ 绿 $\rightarrow$ 黄 $\rightarrow$ 红。可以用逻辑蕴含来表达转换：红变绿为 $R_i \rightarrow G_{i+1}$ （等价于 $\neg R_i \vee G_{i+1}$ ），绿变黄为 $G_i \rightarrow Y_{i+1}$ （等价于 $\neg G_i \vee Y_{i+1}$ ），黄变红为 $Y_i \rightarrow R_{i+1}$ （等价于 $\neg Y_i \vee R_{i+1}$ ）。为了严谨，还得加上**互斥约束**（每一时刻只能亮一个灯）：

$$Unique_i = (R_i \vee Y_i \vee G_i) \wedge (\neg R_i \vee \neg Y_i) \wedge (\neg Y_i \vee \neg G_i) \wedge (\neg R_i \vee \neg G_i)$$

则最终的迁移关系 $Transition(s_i, s_{i+1})$ 就是这些子句的合集：

$$Transition(s_i, s_{i+1}) = T(s_i, s_{i+1}) \wedge Unique_i \wedge Unique_{i+1}$$

假设我们的安全要求是：“绝对不能出现绿灯和黄灯同时亮”，那么编码安全属性 $P(s_i) = \neg(G_i \wedge Y_i)$ 。要找的是反例，所以要编码其否定形式： $\neg P(s_i) = G_i \wedge Y_i$ 。完整的 BMC 公式展开 ($k = 1$ 步) 则为：

$$F_1 = \underbrace{(R_0 \wedge \neg Y_0 \wedge \neg G_0)}_{I(s_0)} \wedge \underbrace{Transition(s_0, s_1)} \wedge \underbrace{(G_0 \wedge Y_0 \vee G_1 \wedge Y_1)}_{\neg P(s_0) \vee \neg P(s_1)}$$

3.3 软件验证方案：CBMC

CBMC 将 C 程序语义映射为布尔约束：

条件分支 (**MUX** 逻辑)：

```
if (cond) x = a; else x = b;
```

$\downarrow x_{new} \leftrightarrow (cond \wedge a) \vee (\neg cond \wedge b)$

循环展开：按边界 k 物理展开为多段 ‘if’ 逻辑，并添加 *Unwinding Assertion*。

工业应用：

- **Linux** 内核：利用 Undertaker 工具检查 Kconfig 配置冲突，识别 1,776 个配置缺陷。

4.1 SAT 与机器学习的深度结合

- **NeuroSAT** (神经求解器): 利用 GNN 将 CNF 建模为“变量-子句”二部图, 学习文字嵌入。
- 强化学习 (RL): 动态调整 CDCL 的分支策略或重启频率 (如 LangSAT 框架)。
- **LLM for Solvers**: 利用大模型驱动代码自我演化 (如 SATLUTION 项目), 在 2025 竞赛中击败人工模型。

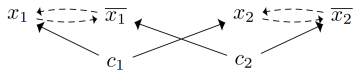


Figure: AI 辅助求解流程示意

4.2 可解释求解与 UNSAT 证明校验

在安全关键领域，仅有“UNSAT”结论是不够的，需要“绝对证据”。

- 证明格式：DRAT (基于单元传播原理)、LRAT (线性时间校验)。
- 前沿方向：分布式在线校验 (On-the-fly Checking)，将校验开销压缩至 42% 以内。

4.3 领域专用与硬件加速

- 专用求解器：针对电路结构的 X-SAT。
- 软硬协同：利用 FPGA 加速并行传播（如 Synopsys ZeBu 系统）。

核心结论

- SAT 已从理论模型演变为数字文明的逻辑算力引擎。
- 现代求解器的成功源于对工业实例“社区结构”的挖掘与分治策略。
- 国产 EDA 工具（如 X-SAT）的突破标志着我国进入全球第一梯队。

未来方向：

- 智能化（AI 自进化）
- 全可信化（自动证明校验）
- 深领域化（电路/安全专用引擎）

感谢各位聆听！

Q & A